

Project Report

Database Management System

**ONLINE COURSE LEARNING PROGRESS
MANAGEMENT SYSTEM**



Submitted by:

Saanvi Eluri(25WU0101114)

R.Bhavya Sree(25WU0101108)

K.Kushal Karthikeya(25WU0101055)

Submitted to:

Dr. Kiran Mayee Adavala

Specialization: AIML

Section: AIML Wolves

(B.Tech 2025-29)

School of Technology

2. Abstract

Online learning platforms deliver content through courses, modules and lessons, and they must track which learners are enrolled, how far each learner has progressed, how they performed in quizzes and whether they earned a certificate. When this information is kept in spreadsheets or scattered files, progress figures become unreliable and reporting is slow and error-prone.

This project presents the design and implementation of a relational database management system for an Online Course Learning Progress Management System. The conceptual design is expressed as an entity-relationship diagram with thirteen entities, converted into a relational schema and normalized to Third Normal Form. The database is implemented in MySQL with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints that enforce the business rules: one enrolment per learner per course, progress between 0 and 100, a limit on quiz attempts, valid scores, and certificates only for completed enrolments.

The database is populated with realistic sample data and queried with joins, aggregate functions, nested queries and a view to produce reports on learner progress, course completion, dropout, quiz performance, certificate eligibility and instructor summaries. A Python (Tkinter) application connected to the database through the MySQL connector supports inserting, viewing and deleting records, recording lesson progress and quiz attempts, issuing certificates and running reports. Testing confirmed that valid operations succeed and that invalid operations are rejected either by the application or by the database constraints.

Keywords: DBMS, MySQL, ER diagram, normalization, 3NF, SQL, Python, Tkinter, e-learning, progress tracking.

3. Introduction and Problem Statement

3.1 Introduction

E-learning has become a primary way of delivering education and training. A typical platform organises content in a hierarchy: a course is divided into modules, and each module contains lessons. Learners enrol in courses, work through lessons, attempt quizzes and finally receive a certificate. Every one of these steps generates data that must be stored accurately and retrieved quickly, both for the learner and for the instructors and administrators who monitor the platform.

A database management system provides the structure needed for this: a well-designed relational schema removes redundancy, constraints protect the quality of the data, and SQL makes it possible to answer questions such as "how far has this learner progressed?" or "which enrolments are eligible for a certificate?" in a single query.

3.2 Problem Statement

Online learning platforms must track the content hierarchy, enrolments, lesson progress, quizzes, attempts, scores and certificates. Without structured data, progress is unreliable: learners can be enrolled twice in the same course, progress values can be entered outside a valid range, quiz attempts can exceed the permitted limit, and certificates can be issued without course completion. The problem addressed in this project is to design and implement a normalized relational database, together with a simple front-end application, that stores this information consistently and supports meaningful reports.

3.3 Business Rules

- A learner can be enrolled in a given course only once (UNIQUE learner and course).
- Lesson completion is a percentage between 0 and 100 (CHECK constraint).
- Each quiz allows a limited number of attempts (max_attempts), and a score can never exceed the quiz maximum (validated by the application).
- An enrolment is marked Completed when every lesson of the course reaches 100 percent.
- A certificate can be issued only for a completed enrolment, and only one certificate exists per enrolment.
- A course, learner or other record that is still referenced by other records cannot be deleted (FOREIGN KEY).

4. Objectives and Scope

4.1 Objectives

- Analyse the requirements and identify the entities, attributes, relationships and users of an online course platform.
- Draw an ER diagram and convert it into a relational schema normalized up to Third Normal Form (3NF).
- Implement the schema in MySQL with primary keys, foreign keys and UNIQUE, NOT NULL, CHECK and DEFAULT constraints.

- Populate every table with at least five realistic records and write SQL for joins, aggregate functions, nested queries and views.
- Build a front-end application connected to the database that supports insertion, deletion and viewing of records.
- Demonstrate data integrity, input validation, transaction awareness, indexing and systematic testing.

4.2 Scope

The project covers the database design and a desktop prototype. The database contains 13 tables, one view and four indexes. The application provides screens for enrolments, learners, courses, lesson progress, quiz attempts, reports and a database browser. Authentication with separate logins for administrators, instructors and learners, online payment, video streaming and automatic quiz marking from answer sheets are outside the scope of the prototype and are listed as future enhancements.

4.3 Intended Users

User	How the system helps
Administrator	Registers learners, publishes courses, monitors enrolments, and issues certificates.
Instructor	Views course enrolment, dropout, quiz performance and instructor summary reports.
Learner	Enrols in courses, records lesson progress, attempts quizzes and receives certificates.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Specification
Operating system	Windows 10 or Windows 11
Database server	MySQL Server 8.x
Database tool	MySQL Workbench 8.x (to run SQL scripts and queries)
Programming language	Python 3 (Tkinter is included in the standard installation)
Database connector	mysql-connector-python
Code editor	Visual Studio Code
Version control	Git and GitHub

5.2 Hardware Requirements

Component	Minimum specification
Processor	Dual-core processor, 2 GHz or faster
Memory (RAM)	4 GB (8 GB recommended)
Storage	1 GB of free disk space
Display	1280 x 800 resolution or higher

6. ER Diagram

The conceptual design contains thirteen entities. The master entities (black headers) hold the course content hierarchy and the people involved; the transaction entities (grey headers) record what learners do. Primary keys are underlined and foreign keys are marked (FK). Labels on the connecting lines show the cardinality of each relationship.

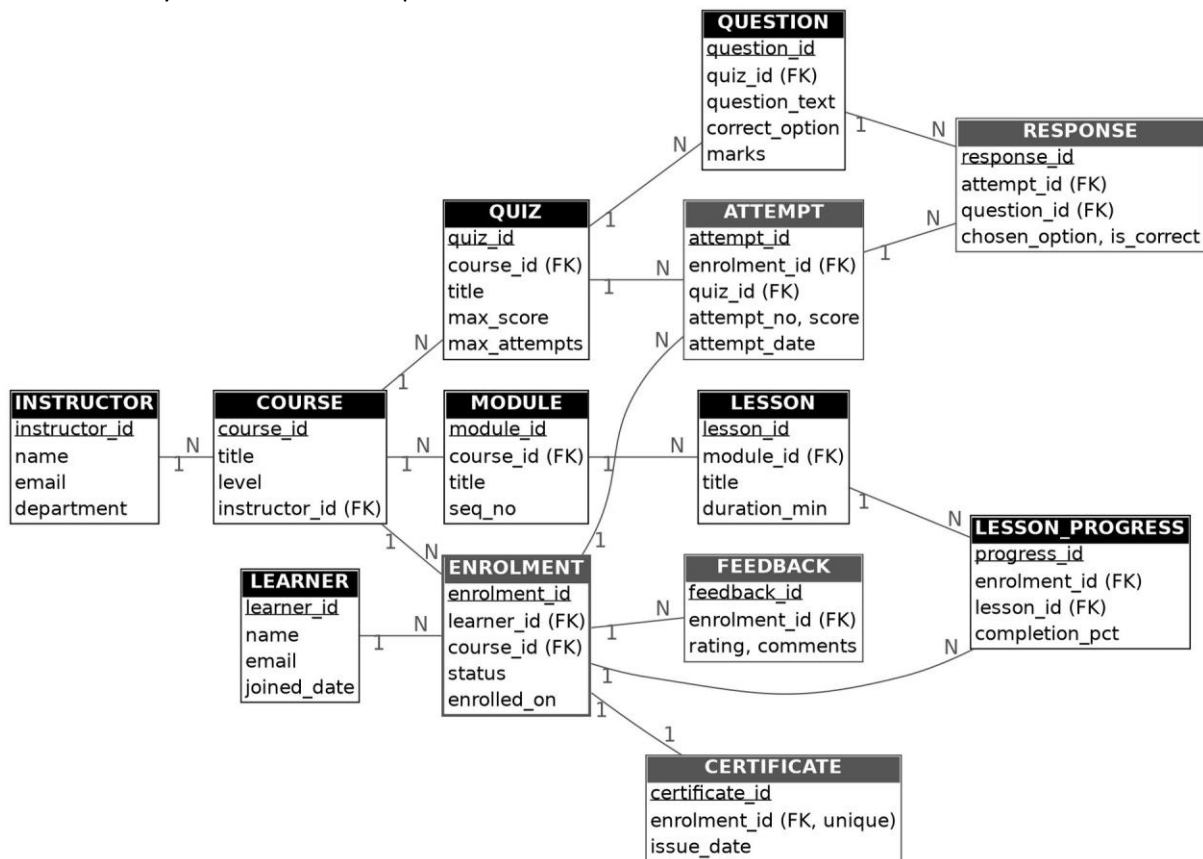


Figure 6.1: ER diagram of the Online Course Learning Progress Management System

6.1 Entities

Entity	Description	Primary key
Instructor	Person who teaches and owns courses	instructor_id
Learner	Registered student of the platform	learner_id
Course	A course offered by one instructor	course_id
Module	Ordered section of a course	module_id
Lesson	Teaching unit inside a module	lesson_id
Enrolment	Links a learner to a course (resolves the many-to-many relationship)	enrolment_id
Lesson_Progress	Completion percentage of one lesson for one enrolment	progress_id
Quiz	Assessment attached to a course	quiz_id

Question	Question belonging to a quiz	question_id
Attempt	One attempt of a quiz by an enrolment, with its score	attempt_id
Response	Answer given to one question within an attempt	response_id
Entity	Description	Primary key
Certificate	Certificate issued for a completed enrolment	certificate_id
Feedback	Rating and comments given by a learner for an enrolment	feedback_id

6.2 Relationships, Cardinality and Participation

Relationship	Cardinality	Participation
Instructor teaches Course	1 : N	Course is total (every course has an instructor); instructor is partial
Course contains Module	1 : N	Module is total; course is partial
Module contains Lesson	1 : N	Lesson is total; module is partial
Learner enrolls in Course	M : N via Enrolment	Enrolment is total on both sides; learner and course are partial
Enrolment has Lesson_Progress for Lesson	1 : N	Lesson_Progress is total; one row per enrolment and lesson
Course has Quiz	1 : N	Quiz is total; course is partial
Quiz has Question	1 : N	Question is total; quiz is partial
Enrolment makes Attempt at Quiz	1 : N	Attempt is total; limited by max_attempts
Attempt contains Response to Question	1 : N	Response is total
Enrolment earns Certificate	1 : 1	Certificate is total; enrolment is partial
Enrolment receives Feedback	1 : N	Feedback is total; enrolment is partial

7. Relational Schema and Normalization

7.1 Relational Schema

The ER diagram is converted to the following relations. Primary keys are shown first; foreign keys are marked FK with the table they reference.

Relation	Attributes
instructor	instructor_id (PK), name, email, department
learner	learner_id (PK), name, email, joined_date
course	course_id (PK), title, level, instructor_id (FK to instructor)
module	module_id (PK), course_id (FK to course), title, seq_no
lesson	lesson_id (PK), module_id (FK to module), title, duration_min
enrolment	enrolment_id (PK), learner_id (FK to learner), course_id (FK to course), status, enrolled_on

lesson_progress	progress_id (PK), enrolment_id (FK to enrolment), lesson_id (FK to lesson), completion_pct
quiz	quiz_id (PK), course_id (FK to course), title, max_score, max_attempts
question	question_id (PK), quiz_id (FK to quiz), question_text, correct_option, marks
attempt	attempt_id (PK), enrolment_id (FK to enrolment), quiz_id (FK to quiz), attempt_no, score, attempt_date
response	response_id (PK), attempt_id (FK to attempt), question_id (FK to question), chosen_option, is_correct
certificate	certificate_id (PK), enrolment_id (FK to enrolment, unique), issue_date
feedback	feedback_id (PK), enrolment_id (FK to enrolment), rating, comments

7.2 Functional Dependencies

Relation	Functional dependencies
instructor	instructor_id -> name, email, department
learner	learner_id -> name, email, joined_date
course	course_id -> title, level, instructor_id
module	module_id -> course_id, title, seq_no; (course_id, seq_no) -> module_id
lesson	lesson_id -> module_id, title, duration_min
enrolment	enrolment_id -> learner_id, course_id, status, enrolled_on; (learner_id, course_id) -> enrolment_id
lesson_progress	progress_id -> enrolment_id, lesson_id, completion_pct; (enrolment_id, lesson_id) -> progress_id
quiz	quiz_id -> course_id, title, max_score, max_attempts
question	question_id -> quiz_id, question_text, correct_option, marks
attempt	attempt_id -> enrolment_id, quiz_id, attempt_no, score, attempt_date; (enrolment_id, quiz_id, attempt_no) -> attempt_id
response	response_id -> attempt_id, question_id, chosen_option, is_correct; (attempt_id, question_id) -> response_id
certificate	certificate_id -> enrolment_id, issue_date; enrolment_id -> certificate_id
Relation	Functional dependencies
feedback	feedback_id -> enrolment_id, rating, comments

7.3 Normalization

To show the normalization process, consider an unnormalized relation in which all information is stored together:

```
LearnerCourse(learner_name, learner_email, course_title, level,
instructor_name, instructor_email, lesson_title, completion_pct)
```

This design repeats the learner, course and instructor details for every lesson, which causes update anomalies (changing an instructor email requires many updates), insertion anomalies (a course cannot be stored until a learner enrolls) and deletion anomalies (deleting the last enrolment loses the course).

First Normal Form (1NF). All attributes hold atomic values, there are no repeating groups, and every relation has a primary key. Each lesson result is stored as its own row in lesson_progress instead of as a list inside the learner record.

Second Normal Form (2NF). The relation is in 1NF and no non-key attribute depends on only part of a composite key. Learner details depend only on learner_id and course details only on course_id, so they are separated into the learner and course tables. The linking tables enrolment and lesson_progress use single-column surrogate primary keys, with the combination of foreign keys protected by a UNIQUE constraint.

Third Normal Form (3NF). The relation is in 2NF and no non-key attribute depends on another non-key attribute. Instructor name, email and department are stored only in the instructor table, and course stores just instructor_id; module and lesson details are likewise held in their own tables. In every relation the non-key attributes depend on the key, the whole key and nothing but the key.

Design note. Two attributes are stored although they can be derived: response.is_correct (from the chosen and correct option) and enrolment.status (from lesson progress). They are kept for fast reporting and are maintained by the application, which sets the status to Completed when all lessons reach 100 percent. This is a deliberate, controlled redundancy.

8. Data Dictionary

The data dictionary lists every table with the data type and constraints of each column. Table-level constraints follow each table.

8.1 instructor

Column	Data type	Constraints
instructor_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(80)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
department	VARCHAR(60)	NOT NULL

8.2 learner

Column	Data type	Constraints
learner_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(80)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
joined_date	DATE	NOT NULL

8.3 course

Column	Data type	Constraints
course_id	INT	PRIMARY KEY, AUTO_INCREMENT
title	VARCHAR(120)	NOT NULL
level	VARCHAR(15)	NOT NULL, DEFAULT 'Beginner'
instructor_id	INT	NOT NULL, FK -> instructor(instructor_id)

Table constraints: CHECK (level IN ('Beginner','Intermediate','Advanced'))

8.4 module

Column	Data type	Constraints
module_id	INT	PRIMARY KEY, AUTO_INCREMENT
course_id	INT	NOT NULL, FK -> course(course_id)
title	VARCHAR(120)	NOT NULL
seq_no	INT	NOT NULL

Table constraints: UNIQUE (course_id, seq_no)

8.5 lesson

Column	Data type	Constraints
lesson_id	INT	PRIMARY KEY, AUTO_INCREMENT
module_id	INT	NOT NULL, FK -> module(module_id)
title	VARCHAR(120)	NOT NULL
duration_min	INT	NOT NULL, DEFAULT 30

Table constraints: CHECK (duration_min > 0)

8.6 enrolment

Column	Data type	Constraints
enrolment_id	INT	PRIMARY KEY, AUTO_INCREMENT
learner_id	INT	NOT NULL, FK -> learner(learner_id)
course_id	INT	NOT NULL, FK -> course(course_id)
status	VARCHAR(12)	NOT NULL, DEFAULT 'Active'
enrolled_on	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table constraints: UNIQUE (learner_id, course_id); CHECK (status IN ('Active','Completed','Dropped'))

8.7 lesson_progress

Column	Data type	Constraints
progress_id	INT	PRIMARY KEY, AUTO_INCREMENT
enrolment_id	INT	NOT NULL, FK -> enrolment(enrolment_id)
lesson_id	INT	NOT NULL, FK -> lesson(lesson_id)
completion_pct	INT	NOT NULL, DEFAULT 0

Table constraints: UNIQUE (enrolment_id, lesson_id); CHECK (completion_pct BETWEEN 0 AND 100); enrolment_id ON DELETE CASCADE

8.8 quiz

Column	Data type	Constraints
quiz_id	INT	PRIMARY KEY, AUTO_INCREMENT
course_id	INT	NOT NULL, FK -> course(course_id)
title	VARCHAR(120)	NOT NULL
max_score	INT	NOT NULL, DEFAULT 10
max_attempts	INT	NOT NULL, DEFAULT 3

Table constraints: CHECK (max_score > 0 AND max_attempts > 0)

8.9 question

Column	Data type	Constraints
question_id	INT	PRIMARY KEY, AUTO_INCREMENT
quiz_id	INT	NOT NULL, FK -> quiz(quiz_id)
question_text	VARCHAR(255)	NOT NULL
correct_option	CHAR(1)	NOT NULL
marks	INT	NOT NULL, DEFAULT 5

Table constraints: CHECK (correct_option IN ('A','B','C','D'))

8.10 attempt

Column	Data type	Constraints
attempt_id	INT	PRIMARY KEY, AUTO_INCREMENT

enrolment_id	INT	NOT NULL, FK -> enrolment(enrolment_id)
quiz_id	INT	NOT NULL, FK -> quiz(quiz_id)
attempt_no	INT	NOT NULL, DEFAULT 1
score	INT	NOT NULL, DEFAULT 0
Column	Data type	Constraints
attempt_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table constraints: UNIQUE (enrolment_id, quiz_id, attempt_no); CHECK (score >= 0); enrolment_id ON DELETE CASCADE

8.11 response

Column	Data type	Constraints
response_id	INT	PRIMARY KEY, AUTO_INCREMENT
attempt_id	INT	NOT NULL, FK -> attempt(attempt_id)
question_id	INT	NOT NULL, FK -> question(question_id)
chosen_option	CHAR(1)	NOT NULL
is_correct	TINYINT(1)	NOT NULL, DEFAULT 0

Table constraints: UNIQUE (attempt_id, question_id); CHECK (chosen_option IN ('A','B','C','D')); attempt_id ON DELETE CASCADE

8.12 certificate

Column	Data type	Constraints
certificate_id	INT	PRIMARY KEY, AUTO_INCREMENT
enrolment_id	INT	NOT NULL, UNIQUE, FK -> enrolment(enrolment_id)
issue_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table constraints: enrolment_id ON DELETE CASCADE

8.13 feedback

Column	Data type	Constraints
feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT
enrolment_id	INT	NOT NULL, FK -> enrolment(enrolment_id)
rating	INT	NOT NULL
comments	VARCHAR(255)	

Table constraints: CHECK (rating BETWEEN 1 AND 5); enrolment_id ON DELETE CASCADE

9. SQL Commands Used

The complete script is provided in schema.sql. The main commands are shown below with their outputs.

9.1 Data Definition Language (DDL) Creating

the database

```
DROP DATABASE IF EXISTS course_progress;
CREATE DATABASE course_progress CHARACTER SET utf8mb4;
USE course_progress;
```

Creating tables with constraints

```
CREATE TABLE course (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(120) NOT NULL,
    level VARCHAR(15) NOT NULL DEFAULT 'Beginner',
    instructor_id INT NOT NULL,
    CONSTRAINT chk_course_level CHECK (level IN
        ('Beginner', 'Intermediate', 'Advanced')),
    FOREIGN KEY (instructor_id) REFERENCES instructor(instructor_id)
);

CREATE TABLE enrolment (
    enrolment_id INT AUTO_INCREMENT PRIMARY KEY,
    learner_id INT NOT NULL,
    course_id INT NOT NULL,
    status VARCHAR(12) NOT NULL DEFAULT 'Active',
    enrolled_on DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (learner_id, course_id),
    CONSTRAINT chk_enrol_status CHECK
        (status IN ('Active', 'Completed', 'Dropped')),
    FOREIGN KEY (learner_id) REFERENCES learner(learner_id),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

CREATE TABLE lesson_progress (
    progress_id INT AUTO_INCREMENT PRIMARY KEY,
    enrolment_id INT NOT NULL,
    lesson_id INT NOT NULL,
    completion_pct INT NOT NULL DEFAULT 0,
    UNIQUE (enrolment_id, lesson_id),
    CONSTRAINT chk_progress_pct CHECK (completion_pct BETWEEN 0 AND 100),
    FOREIGN KEY (enrolment_id) REFERENCES enrolment(enrolment_id)
        ON DELETE CASCADE,
    FOREIGN KEY (lesson_id) REFERENCES lesson(lesson_id)
);
```

Indexes and view

```
CREATE INDEX idx_learner_name ON learner(name);
CREATE INDEX idx_course_title ON course(title);
CREATE INDEX idx_progress_lesson ON lesson_progress(lesson_id);
CREATE INDEX idx_attempt_quiz ON attempt(quiz_id);
```

```
CREATE VIEW v_learner_progress AS
SELECT e.enrolment_id, l.name AS learner, c.title AS course, e.status,
    ROUND(COALESCE(SUM(lp.completion_pct), 0) /
        (SELECT COUNT(*) FROM lesson ls JOIN module m ON m.module_id = ls.module_id
            WHERE m.course_id = c.course_id), 1) AS progress_pct
FROM enrolment e
```

```
JOIN learner l ON l.learner_id = e.learner_id
JOIN course c ON c.course_id = e.course_id
LEFT JOIN lesson_progress lp ON lp.enrolment_id = e.enrolment_id
GROUP BY e.enrolment_id, l.name, c.course_id, c.title, e.status;
```

Output: DESCRIBE enrolment

Field	Type	Null	Key	Default	Extra
enrolment_id	int(11)	NO	PRI		auto_increment
learner_id	int(11)	NO	MUL		
course_id	int(11)	NO	MUL		
status	varchar(12)	NO		Active	
enrolled_on	datetime	NO		current_timestamp()	

Output: row count of each table after loading sample data

Table	Rows	Table	Rows
instructor	5	quiz	5
learner	6	question	10
course	5	attempt	7
module	10	response	8
lesson	20	certificate	5
enrolment	9	feedback	5
lesson_progress	36		

A view is also created, so SHOW TABLES lists 13 tables and the view v_learner_progress (14 entries).

9.2 Data Manipulation Language (DML)

INSERT

```
INSERT INTO learner (name, email, joined_date) VALUES
('Anita Sharma', 'anita.sharma@mail.com', '2026-06-12'),
('Rahul Verma', 'rahul.verma@mail.com', '2026-06-15');

INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3);
```

Output of the last statement: 1 row affected; the number of rows in enrolment increased from 9 to 10.

SELECT

```
SELECT learner_id, name, email, joined_date FROM learner;
```

learner_id	name	email	joined_date
1	Anita Sharma	anita.sharma@mail.com	2026-06-12
2	Rahul Verma	rahul.verma@mail.com	2026-06-15

3	Meera Iyer	meera.iyer@mail.com	2026-06-20
4	Karan Singh	karan.singh@mail.com	2026-07-02
5	Priya Reddy	priya.reddy@mail.com	2026-07-10
6	Arjun Patel	arjun.patel@mail.com	2026-07-18

```
SELECT enrolment_id, learner_id, course_id, status FROM enrolment;
```

enrolment_id	learner_id	course_id	status
1	1	1	Completed
2	2	1	Active
3	3	2	Completed
4	4	3	Completed
5	5	1	Dropped
6	6	4	Completed
7	1	2	Active
8	2	3	Completed
9	4	1	Completed

UPDATE

```
UPDATE enrolment SET status = 'Completed' WHERE enrolment_id = 7;
```

Output: 1 row affected. The application runs this statement automatically when every lesson of the course reaches 100 percent.

DELETE

```
DELETE FROM enrolment WHERE enrolment_id = 10;
```

Output: 1 row affected; the number of rows in enrolment decreased from 10 to 9. Deleting an enrolment also removes its lesson progress, attempts, responses, certificate and feedback because of ON DELETE CASCADE.

10. Queries with Outputs

The following queries demonstrate joins, aggregate functions, GROUP BY with HAVING, nested queries and the view. They cover the required reports: learner progress, course completion, dropout, quiz performance, certificate eligibility and instructor summary.

Query 1: Learner progress (join and aggregate)

Average lesson progress of every learner in each enrolled course, joining four tables.

```
SELECT l.name AS learner, c.title AS course,
       ROUND(AVG(lp.completion_pct), 1) AS progress
FROM learner l
JOIN enrolment e ON e.learner_id = l.learner_id
JOIN course c ON c.course_id = e.course_id
JOIN lesson_progress lp ON lp.enrolment_id = e.enrolment_id
```

```
GROUP BY l.name, c.title;
```

Output:

learner	course	progress
Anita Sharma	Data Science Introduction	40.0
Anita Sharma	Python Basics	100.0
Arjun Patel	Java Object-Oriented Programming	100.0
Karan Singh	Python Basics	100.0
Karan Singh	Web Design Fundamentals	100.0
Meera Iyer	Data Science Introduction	100.0
Priya Reddy	Python Basics	35.0
Rahul Verma	Python Basics	62.5
Rahul Verma	Web Design Fundamentals	100.0

Query 2: Learners per course (LEFT JOIN and COUNT)

Number of learners enrolled in each course; courses without learners are included.

```
SELECT c.title, COUNT(e.enrolment_id) AS learners
FROM course c LEFT JOIN enrolment e ON e.course_id = c.course_id
GROUP BY c.title;
```

Output:

title	learners
Python Basics	4
Data Science Introduction	2
Web Design Fundamentals	2
Java Object-Oriented Programming	1
Statistics for Engineers	0

Query 3: Learners who scored at least 5 out of 10 (nested query) Uses

subqueries to find learners with a passing quiz attempt.

```
SELECT name FROM learner WHERE learner_id IN (
    SELECT learner_id FROM enrolment WHERE enrolment_id IN (
        SELECT enrolment_id FROM attempt WHERE score >= 5));
```

Output:

name
Anita Sharma
Arjun Patel
Karan Singh
Meera Iyer

Rahul Verma

Query 4: View of progress per enrolment

Reads from the view v_learner_progress, which measures progress against all lessons of the course.

```
SELECT enrolment_id, learner, course, status, progress_pct
FROM v_learner_progress;
```

Output:

enrolment_id	learner	course	status	progress_pct
1	Anita Sharma	Python Basics	Completed	100.0
2	Rahul Verma	Python Basics	Active	62.5
3	Meera Iyer	Data Science Introduction	Completed	100.0
4	Karan Singh	Web Design Fundamentals	Completed	100.0
5	Priya Reddy	Python Basics	Dropped	35.0
6	Arjun Patel	Java Object-Oriented Programming	Completed	100.0
7	Anita Sharma	Data Science Introduction	Active	40.0
8	Rahul Verma	Web Design Fundamentals	Completed	100.0
9	Karan Singh	Python Basics	Completed	100.0

Query 5: Popular courses (GROUP BY with HAVING)

Courses with at least two enrolments.

```
SELECT c.title, COUNT(*) AS enrolments
FROM course c JOIN enrolment e ON e.course_id = c.course_id
GROUP BY c.title HAVING COUNT(*) >= 2;
```

Output:

title	enrolments
Python Basics	4
Data Science Introduction	2
Web Design Fundamentals	2

Query 6: Certificate eligibility

Completed enrolments for which no certificate has been issued yet.

```
SELECT e.enrolment_id, l.name AS learner, c.title AS course
FROM enrolment e
JOIN learner l ON l.learner_id = e.learner_id
JOIN course c ON c.course_id = e.course_id
WHERE e.status = 'Completed' AND NOT EXISTS
  (SELECT 1 FROM certificate ce WHERE ce.enrolment_id = e.enrolment_id);
```

Output:

enrolment_id	learner	course
8	Rahul Verma	Web Design Fundamentals

Query 7: Quiz performance

Attempts, average score and best score for each quiz.

```
SELECT q.title AS quiz, COUNT(a.attempt_id) AS attempts,
       ROUND(AVG(a.score), 1) AS avg_score, MAX(a.score) AS best
FROM quiz q JOIN attempt a ON a.quiz_id = q.quiz_id
GROUP BY q.title;
```

Output:

quiz	attempts	avg_score	best
Quiz: Data Science	1	5.0	5
Quiz: Java OOP	1	5.0	5
Quiz: Python Basics	4	6.3	10
Quiz: Web Design	1	10.0	10

Query 8: Course completion and dropout

Total, dropped and completed enrolments for each course.

```
SELECT c.title AS course, COUNT(*) AS total,
       SUM(e.status = 'Dropped') AS dropped,
       SUM(e.status = 'Completed') AS completed
FROM course c JOIN enrolment e ON e.course_id = c.course_id
GROUP BY c.title;
```

Output:

course	total	dropped	completed
Data Science Introduction	2	0	1
Java Object-Oriented Programming	1	0	1
Python Basics	4	1	2
Web Design Fundamentals	2	0	2

Query 9: Instructor summary

Courses taught and total enrolments for every instructor.

```
SELECT i.name AS instructor, COUNT(DISTINCT c.course_id) AS courses,
       COUNT(e.enrolment_id) AS enrolments
FROM instructor i
JOIN course c ON c.instructor_id = i.instructor_id
LEFT JOIN enrolment e ON e.course_id = c.course_id
GROUP BY i.name;
```

Output:

instructor	courses	enrolments
Dr. Ananya Rao	1	4
Dr. Farah Khan	1	0
Dr. Kavita Nair	1	2
Prof. Suresh Kulkarni	1	1

Prof. Vikram Mehta	1	2
--------------------	---	---

11. UI Design and Screenshots

11.1 Design

The user interface is a desktop application built with Python Tkinter using a light theme. A tabbed layout gives one screen per area of the system, and each screen has the same structure: an input form at the top for insertion, a table below for viewing records, and buttons for refreshing and deleting the selected row. At the bottom of the window, an activity panel shows the exact SQL statement executed and the number of rows in every changed table before and after the action, so each change can be seen to be reflected in the database.

Screen	Purpose
Enrolments	Enrol a learner in a course (insert), view enrolments, delete an enrolment
Learners	Register a learner, view learners, delete a learner
Courses	Publish a course for an instructor, view courses, delete a course
Lesson Progress	Record or update completion percentage of a lesson; view and delete records
Quiz Attempts	Record a quiz attempt with validation of score and attempt limit; view and delete
Reports	Learner progress, course summary and certificate eligibility; issue a certificate
Database Browser	Shows the contents of any table directly from MySQL

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

Enrolments

Learners

Courses

Lesson Progress

Quiz Attempts

Reports

Database Browser

Enrol a learner in a course (INSERT)

Learner

Course

Enrol

Enrolments (VIEW)

Refresh

Delete selected

Enrolment Id	Learner	Course	Status	Enrolled On
9	Karan Singh	Python Basics	Completed	2026-08-10 10:00:00
8	Rahul Verma	Web Design Fundamentals	Completed	2026-08-04 10:00:00
7	Anita Sharma	Data Science Introduction	Active	2026-08-01 10:00:00
6	Arjun Patel	Java Object-Oriented Programmi	Completed	2026-07-20 10:00:00
5	Priya Reddy	Python Basics	Dropped	2026-07-12 10:00:00
4	Karan Singh	Web Design Fundamentals	Completed	2026-07-08 10:00:00
3	Meera Iyer	Data Science Introduction	Completed	2026-07-05 10:00:00
2	Rahul Verma	Python Basics	Active	2026-07-03 10:00:00
1	Anita Sharma	Python Basics	Completed	2026-07-01 10:00:00

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

Figure 11.1: Enrolments screen before insertion (9 enrolments)

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

Enrolments

Learners

Courses

Lesson Progress

Quiz Attempts

Reports

Database Browser

Enrol a learner in a course (INSERT)

Learner

5 - Priya Reddy

Course

3 - Web Design Fundamentals

Enrol

Enrolments (VIEW)

Refresh

Delete selected

Enrolment Id	Learner	Course	Status	Enrolled On
10	Priya Reddy	Web Design Fundamentals	Active	2026-10-06 18:59:15
9	Karan Singh	Python Basics	Completed	2026-08-10 10:00:00
8	Rahul Verma	Web Design Fundamentals	Completed	2026-08-04 10:00:00
7	Anita Sharma	Data Science Introduction	Active	2026-08-01 10:00:00
6	Arjun Patel	Java Object-Oriented Programmi	Completed	2026-07-20 10:00:00
5	Priya Reddy	Python Basics	Dropped	2026-07-12 10:00:00
4	Karan Singh	Web Design Fundamentals	Completed	2026-07-08 10:00:00
3	Meera Iyer	Data Science Introduction	Completed	2026-07-05 10:00:00
2	Rahul Verma	Python Basics	Active	2026-07-03 10:00:00
1	Anita Sharma	Python Basics	Completed	2026-07-01 10:00:00

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment

SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)

Row counts enrolment: 9 -> 10

Figure 11.2: After inserting an enrolment; the activity panel shows INSERT and enrolment 9 -> 10

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

Enrolments

Learners

Courses

Lesson Progress

Quiz Attempts

Reports

Database Browser

Enrol a learner in a course (INSERT)

Learner

5 - Priya Reddy

Course

3 - Web Design Fundamentals

Enrol

Enrolments (VIEW)

Refresh

Delete selected

Enrolment Id	Learner	Course	Status	Enrolled On
9	Karan Singh	Python Basics	Completed	2026-08-10 10:00:00
8	Rahul Verma	Web Design Fundamentals	Completed	2026-08-04 10:00:00
7	Anita Sharma	Data Science Introduction	Active	2026-08-01 10:00:00
6	Arjun Patel	Java Object-Oriented Programmi	Completed	2026-07-20 10:00:00
5	Priya Reddy	Python Basics	Dropped	2026-07-12 10:00:00
4	Karan Singh	Web Design Fundamentals	Completed	2026-07-08 10:00:00
3	Meera Iyer	Data Science Introduction	Completed	2026-07-05 10:00:00
2	Rahul Verma	Python Basics	Active	2026-07-03 10:00:00
1	Anita Sharma	Python Basics	Completed	2026-07-01 10:00:00

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment

SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)

Row counts enrolment: 9 -> 10

[18:59:15] DELETE enrolment #10

SQL: DELETE FROM enrolment WHERE enrolment_id = 10 -> 1 row(s)

Row counts enrolment: 10 -> 9

Figure 11.3: After deleting the new enrolment; the panel shows DELETE and enrolment 10 -> 9

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

Enrolments

Learners

Courses

Lesson Progress

Quiz Attempts

Reports

Database Browser

Register a learner (INSERT)

Name

Email

Joined (YYYY-MM-DD)

2026-10-06

Add learner

Learners (VIEW)

Refresh

Delete selected

Learner Id	Name	Email	Joined Date
6	Arjun Patel	arjun.patel@mail.com	2026-07-18
5	Priya Reddy	priya.reddy@mail.com	2026-07-10
4	Karan Singh	karan.singh@mail.com	2026-07-02
3	Meera Iyer	meera.iyer@mail.com	2026-06-20
2	Rahul Verma	rahul.verma@mail.com	2026-06-15
1	Anita Sharma	anita.sharma@mail.com	2026-06-12

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment

SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)

Row counts enrolment: 9 -> 10

[18:59:15] DELETE enrolment #10

SQL: DELETE FROM enrolment WHERE enrolment_id = 10 -> 1 row(s)

Row counts enrolment: 10 -> 9

Figure 11.4: Learners screen

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

Enrolments

Learners

Courses

Lesson Progress

Quiz Attempts

Reports

Database Browser

Publish a course (INSERT)

Title

Level

Beginner

Instructor

Publish course

Courses (VIEW)

Refresh

Delete selected

Course Id	Title	Level	Instructor
5	Statistics for Engineers	Advanced	Dr. Farah Khan
4	Java Object-Oriented Programming	Intermediate	Prof. Suresh Kulkarni
3	Web Design Fundamentals	Beginner	Dr. Kavita Nair
2	Data Science Introduction	Intermediate	Prof. Vikram Mehta
1	Python Basics	Beginner	Dr. Ananya Rao

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment

SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)

Row counts enrolment: 9 -> 10

[18:59:15] DELETE enrolment #10

SQL: DELETE FROM enrolment WHERE enrolment_id = 10 -> 1 row(s)

Row counts enrolment: 10 -> 9

Figure 11.5: Courses screen

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

EnrolmentsLearnersCoursesLesson ProgressQuiz AttemptsReportsDatabase Browser

Record lesson progress (INSERT / UPDATE)

Enrolment

Lesson

Completion %100

Save progress

Lesson progress (VIEW)

Refresh

Delete selected

Progress Id	Learner	Course	Lesson	Completion Pct
36	Karan Singh	Python Basics	Writing Functions	100
35	Karan Singh	Python Basics	Loops	100
34	Karan Singh	Python Basics	Variables and Types	100
33	Karan Singh	Python Basics	Installing Python	100
32	Rahul Verma	Web Design Fundamentals	Flexbox and Grid	100
31	Rahul Verma	Web Design Fundamentals	CSS Selectors	100
30	Rahul Verma	Web Design Fundamentals	Forms	100
29	Rahul Verma	Web Design Fundamentals	HTML Structure	100
28	Anita Sharma	Data Science Introduction	Model Evaluation	0
27	Anita Sharma	Data Science Introduction	Linear Regression	0
26	Anita Sharma	Data Science Introduction	Reading CSV Files	60
25	Anita Sharma	Data Science Introduction	Lists and Dicts	100

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment
SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)
Row counts enrolment: 9 -> 10
[18:59:15] DELETE enrolment #10
SQL: DELETE FROM enrolment WHERE enrolment_id = 10 -> 1 row(s)
Row counts enrolment: 10 -> 9

Figure 11.6: Lesson Progress screen

Online Course Learning Progress Management

Connected to MySQL course_progress @ localhost

EnrolmentsLearnersCoursesLesson ProgressQuiz AttemptsReportsDatabase Browser

Record a quiz attempt (INSERT)

Enrolment

Quiz

Score0

Record attempt

Quiz attempts (VIEW)

Refresh

Delete selected

Attempt Id	Learner	Quiz	Attempt No	Score	Attempt Date
7	Arjun Patel	Quiz: Java OOP	1	5 / 10	2026-08-05 11:00:00
6	Priya Reddy	Quiz: Python Basics	1	0 / 10	2026-07-22 11:00:00
5	Karan Singh	Quiz: Web Design	1	10 / 10	2026-07-25 11:00:00
4	Meera Iyer	Quiz: Data Science	1	5 / 10	2026-07-20 11:00:00
3	Rahul Verma	Quiz: Python Basics	2	10 / 10	2026-07-14 11:00:00
2	Rahul Verma	Quiz: Python Basics	1	5 / 10	2026-07-12 11:00:00
1	Anita Sharma	Quiz: Python Basics	1	10 / 10	2026-07-10 11:00:00

Database activity: SQL executed and row counts

Ready. Every change you make is committed to MySQL and shown here.

[18:59:15] INSERT enrolment
SQL: INSERT INTO enrolment (learner_id, course_id) VALUES (5, 3) -> 1 row(s)
Row counts enrolment: 9 -> 10
[18:59:15] DELETE enrolment #10
SQL: DELETE FROM enrolment WHERE enrolment_id = 10 -> 1 row(s)
Row counts enrolment: 10 -> 9

Figure 11.7: Quiz Attempts screen

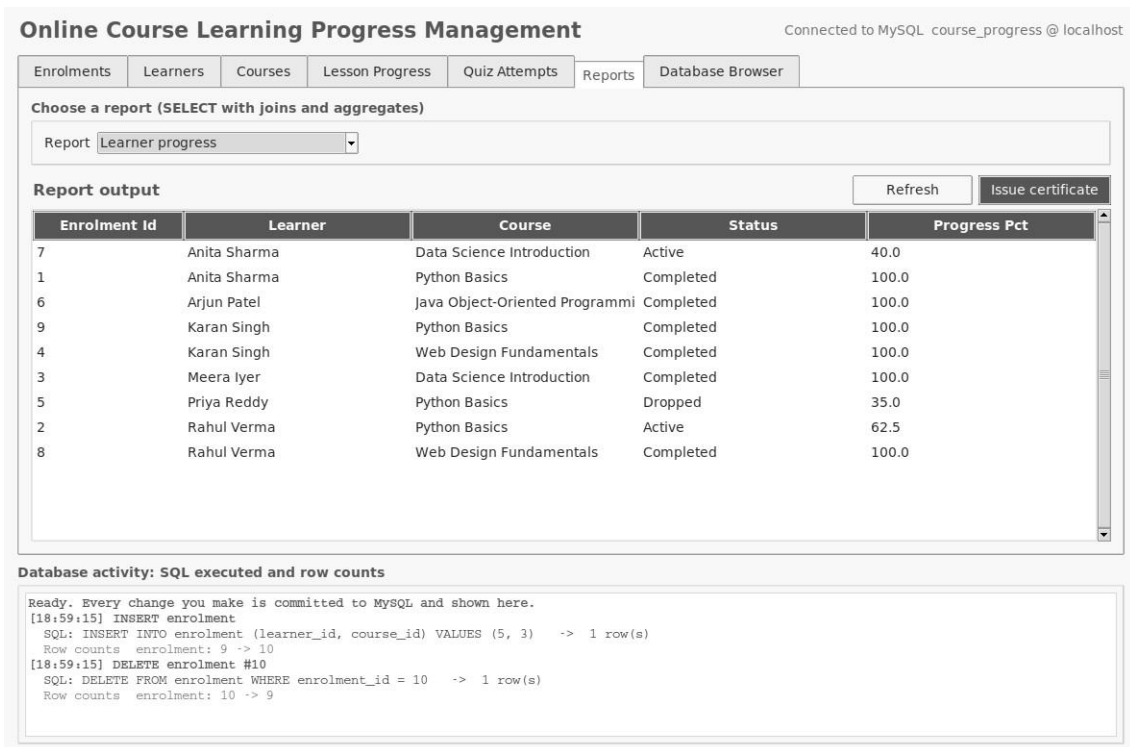


Figure 11.8: Reports screen (learner progress report)

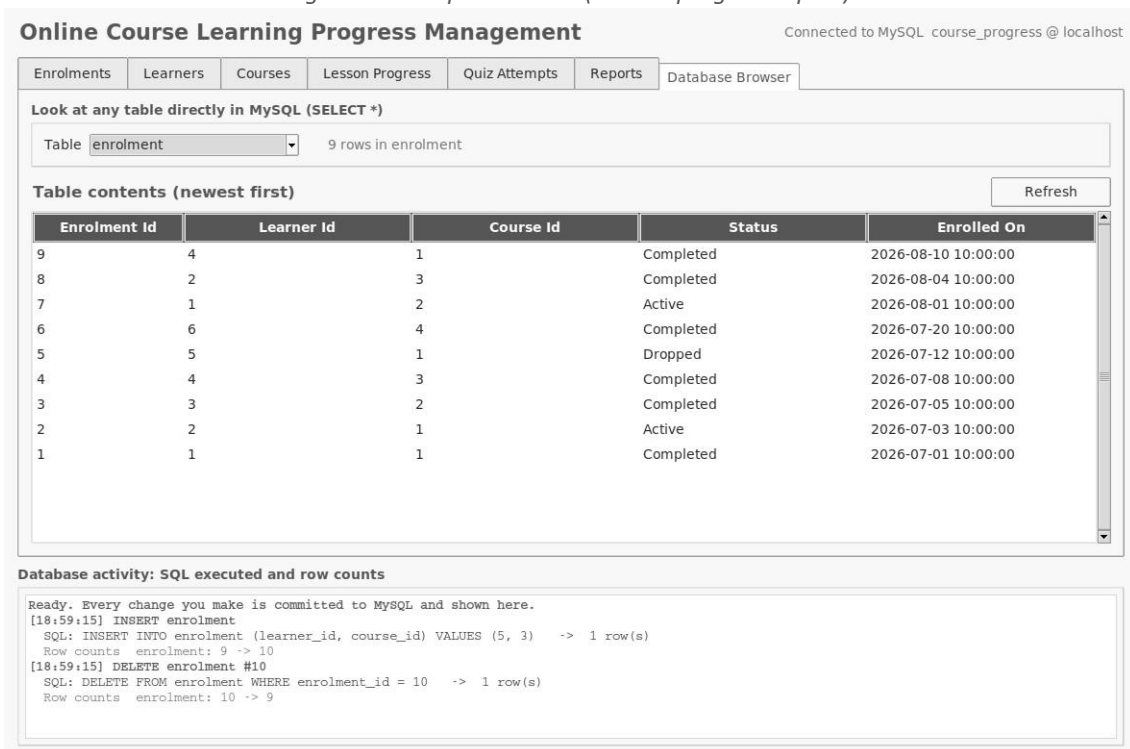


Figure 11.9: Database Browser showing a table read directly from MySQL

12. Implementation Details

12.1 Technology Stack

Layer	Technology
-------	------------

Presentation (UI)	Python 3 with Tkinter and ttk widgets (app.py)
Business logic and data access	Python module db.py: validation rules, SQL execution, transactions
Database connectivity	mysql-connector-python
Database	MySQL 8.x (schema.sql creates tables, constraints, indexes, view and sample data)

12.2 Database Connectivity

The application connects to MySQL using the connection settings in db.py. Autocommit is switched on so that reads always see the latest committed data, and every write runs inside an explicit transaction that is committed on success and rolled back on any error.

```
import mysql.connector
CONFIG = {"host": "localhost", "port": 3306, "user": "root",
         "password": "<password>", "database": "course_progress"}
class Db:
    def __init__(self):
        self.conn
= mysql.connector.connect(**CONFIG)
    self.conn.autocommit = True

    @contextmanager
    def
tx(self):
self.conn.start_transaction()
try:
    yield
    self.conn.commit()
except Exception:
self.conn.rollback()
    raise
```

12.3 Key Code Snippets

Insertion with a parameterized query

User input is passed as parameters (%s placeholders) and never joined into the SQL string, which prevents SQL injection.

```
def enrol(self, learner_id,
course_id):
    with self.tx():
self.execute(
    "INSERT INTO enrolment (learner_id, course_id) VALUES (%s, %s)",
    (learner_id, course_id))
```

Saving progress with validation and automatic completion

```
def save_progress(self, enrolment_id, lesson_id, pct):
    if not 0 <= pct <= 100:
        raise ValueError("Progress must be
between 0 and 100.")
    ... # checks: enrolment exists, not dropped,
lesson is in the course
    with self.tx():
        self.execute("INSERT
INTO lesson_progress (enrolment_id, lesson_id,
completion_pct)
VALUES (%s, %s, %s) ON DUPLICATE KEY UPDATE
completion_pct = VALUES(completion_pct)", (...))
    if done == total and status == "Active":
self.execute("UPDATE enrolment SET status = 'Completed'
WHERE enrolment_id = %s", (enrolment_id,))
```

Enforcing the attempt limit

```
used = self.one("SELECT COUNT(*) FROM attempt WHERE enrolment_id = %s
AND quiz_id = %s", (enrolment_id, quiz_id))[0] if used >= max_attempts:      raise
ValueError(f"Attempt limit reached ({max_attempts} attempts allowed).")
```

Error handling in the user interface

Every action goes through one function that records the row counts before and after the change. Application-level validation errors are shown as warnings, and errors raised by MySQL (duplicate key, foreign key, check constraint) are translated into readable messages and written to the activity panel as rejected operations.

```
def run_action(self, label, fn):
    before = self.db.counts()
    try:
        fn()
    except ValueError as e:
        messagebox.showwarning("Not allowed", str(e)); return False
    except Error as e:
        messagebox.showerror("Database error", friendly(e));
    return False
    after = self.db.counts()
    # log the SQL executed and the tables whose row counts changed
```

12.4 Integrity, Validation and Indexing

- Data integrity is enforced by PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints in the database itself.
- Input validation in the application checks required fields, the email format, the date format, the progress range, the score range and the attempt limit before any SQL is sent.
- Transaction awareness: each write runs inside a transaction that is rolled back on failure, so the database never keeps a partial change.
- Indexes on learner name, course title, lesson_progress.lesson_id and attempt.quiz_id speed up frequently searched columns.

13. Testing

Testing covered valid operations, invalid input rejected by the application, and invalid data rejected by the database constraints. Each test case was executed against the project database and the actual result was recorded.

ID	Test case	Expected result	Actual result	Result
TC1	Insert a new enrolment (Priya Reddy, Web Design Fundamentals)	Row is inserted	Inserted: enrolment 9->10	Pass
TC2	Insert the same learner and course again	Rejected by UNIQUE constraint	Rejected: duplicate entry (error 1062); no change	Pass
TC3	Set completion_pct to 120 using SQL	Rejected by CHECK constraint	Rejected: constraint chk_progress_pct violated	Pass
TC4	Enter progress 150 in the application	Rejected by validation	Message: Progress must be between 0 and 100.	Pass
TC5	Delete a course that has modules and enrolments	Blocked by FOREIGN KEY	Rejected: cannot delete a parent row (error 1451)	Pass

TC6	Delete a learner who has enrolments	Blocked by FOREIGN KEY	Rejected: cannot delete a parent row (error 1451)	Pass
TC7	Record progress for a lesson of a different course	Rejected by validation	Message: That lesson does not belong to the enrolled course.	Pass
TC8	Record the 3rd attempt of a quiz that allows 3	Accepted	Inserted: attempt 7->8	Pass
TC9	Record a 4th attempt of the same quiz	Rejected: attempt limit	Message: Attempt limit reached (3 attempts allowed).	Pass
TC10	Record a score of 15 for a quiz with maximum 10	Rejected by validation	Message: Score must be between 0 and 10 for this quiz.	Pass
TC11	Record progress for a dropped enrolment	Rejected by validation	Message: This enrolment was dropped; progress cannot be recorded.	Pass
TC12	Issue a certificate for an active enrolment	Rejected: not completed	Message: Certificate can be issued only for a completed enrolment.	Pass
TC13	Set all lessons of an enrolment to 100%	Status becomes Completed	Status changed to Completed	Pass
TC14	Issue a certificate for the completed enrolment	Certificate inserted	Inserted: certificate 5->6	Pass
TC15	Delete an enrolment that has related records	Related rows deleted (CASCADE)	enrolment 10->9, lesson_progress 36->32, attempt 8->7, response 8->6, certificate 6->5, feedback 5->4	Pass
TC16	Insert, view and delete from the user interface	Each change appears in the database	Confirmed in activity panel and Database Browser (Figures 11.1 to 11.3)	Pass

All test cases passed. The tests show that the rules of the system are enforced at two levels: the application validates input and gives clear messages, and the database constraints act as a final safeguard even if invalid data is sent directly with SQL.

14. Conclusion and Future Enhancements

14.1 Conclusion

The project delivered a normalized relational database and a working front-end for tracking learner progress on an online course platform. The design was modelled with an ER diagram of thirteen entities and converted to a schema in Third Normal Form. The MySQL implementation enforces the business rules through constraints, and the sample queries produce the required reports on progress, completion, dropout, quiz performance, certificate eligibility and instructors. The Python application connects to the database, supports insertion, deletion and viewing, validates input and shows every change reflected in the database. Systematic testing confirmed correct behaviour for valid and invalid operations.

14.2 Future Enhancements

- Role-based login so that administrators, instructors and learners see only their own screens.
- A web interface so that learners can use the system from a browser.
- Automatic marking of quizzes from the stored responses and computation of the attempt score.
- Video and reading progress tracking inside each lesson.
- Automatic generation of certificates as PDF files and email notification.

- Charts and dashboards for instructors, and stored procedures and triggers for completion logic.

15. References

- [1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th Edition, Pearson, 2016.
- [2] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, 7th Edition, McGraw-Hill, 2019.
- [3] MySQL 8.0 Reference Manual, Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/>
- [4] MySQL Connector/Python Developer Guide, Oracle Corporation.
<https://dev.mysql.com/doc/connector-python/en/>
- [5] Python documentation: tkinter, Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>
- [6] DBMS course material and laboratory manual, School of Technology, 2025-2026.

16. Contribution of Each Member

The work was divided among the three members of the group. The responsibilities are recorded below.

Member name	Roll number	Contribution
Saanvi Eluri	25WU0101114	Database, UI, report
R. Bhavya Sree	25WU0101108	SQL testing, GitHub upload
K. Kushal Karthikeya	25WU0101055	UI testing, report proofreading

17. Appendix: GitHub Repository Link

Repository: <https://github.com/saanvieluri/DBMS-Course-Project> Structure
of the repository:

```
DBMS-Course-Project/
  README.md           project title, member names and roll numbers
  Presentation/       PPT or PDF, ER diagram image, schema.sql,
                      UI source code (app.py, db.py), UI screenshots
  Project-Report/     project report (PDF)
```